Fundamentals of API Development

API Ecosystem

Day 4 - Session 2

KavinSchool

What We'll Cover

1. API-first in practice and edge component responsibilities
2. The layered test strategy
3. Contract testing: catching drift automatically
4. Security within the CI/CD ecosystem (briefly)
5. The case for deep API knowledge
6. Demos: contract drift, test pyramid
7. Card applications capstone, review, course recap

API-First, in Practice

From spec to running code

KavinSchool

Codegen: One Spec, Two Outputs

OpenAPI Generator reads the same spec file you wrote in Lab 1.2 and produces:

- **Server stubs**: Spring Boot controller scaffolding that matches the contract
- **Client SDKs**: typed Java clients for consumers, generated instead of hand-written

> Design and implementation stay in sync automatically, because both are generated from the same artifact.

Parallel-Track Development

Once a contract is agreed, frontend and backend stop waiting on each other:

- Frontend teams build against mocked responses generated from the spec
- Backend teams implement the real logic against that same spec
- Organizations report up to 50% less development time from this decoupling

Parallel-Track Development (Fintech)

REAL-WORLD SCENARIO

Fintech (Navy Federal — Mobile Strategy):

- Navy Federal's digital teams generate iOS, Android, and web client SDKs from a single OpenAPI spec.
- This allows mobile UI development to happen in parallel with backend ledger integrations, significantly reducing time-to-market for new banking features.

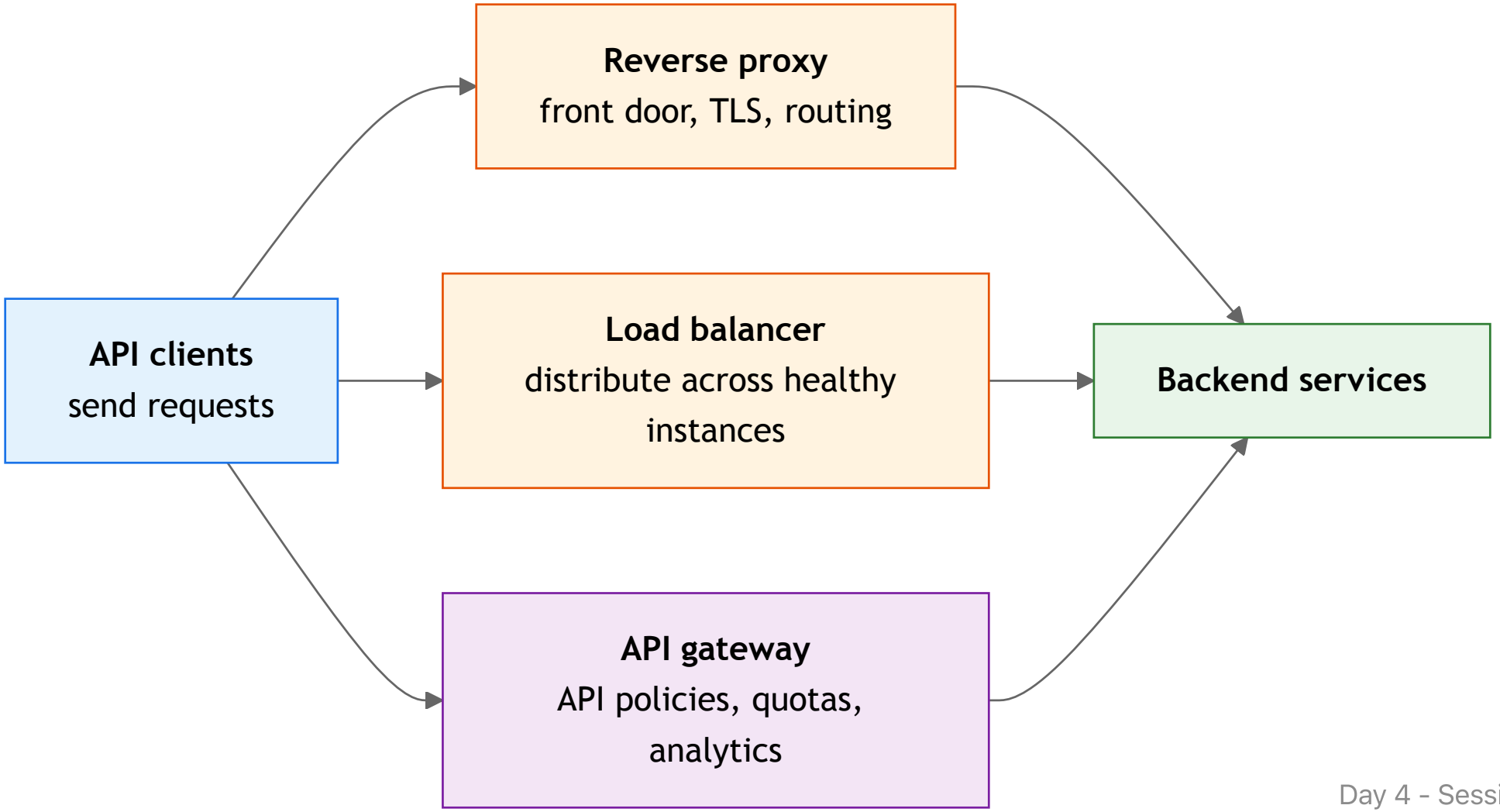
Parallel-Track Development (Retail)

REAL-WORLD SCENARIO

Retail (Target — Point of Sale):

- Target generates Point-of-Sale (POS) client stubs directly from their inventory API specifications.
- This parallel-track approach cuts the integration time for new store checkout features by half, as the POS terminals can be developed against a stable contract.

Reverse Proxy, Load Balancer, or API Gateway?



The Layered Test Strategy

Cheapest and fastest first, always

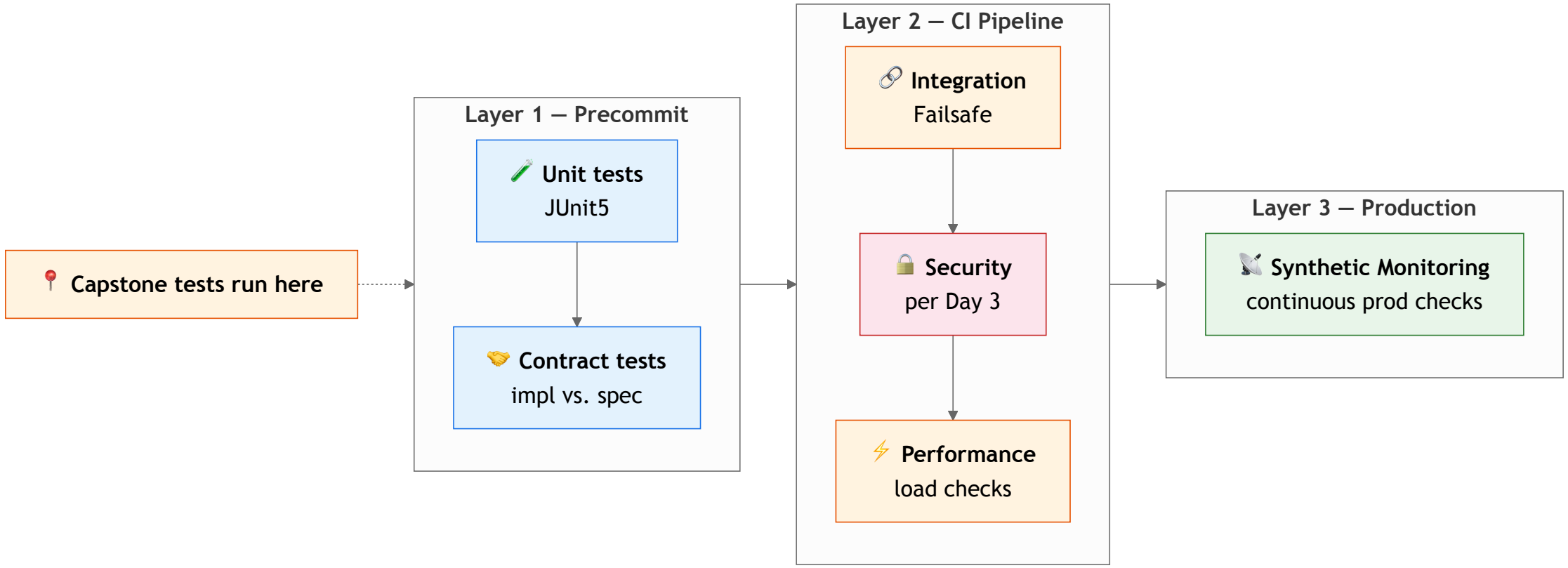
KavinSchool

Three Layers, in Order

1. **Precommit**: unit tests (JUnit5) and contract tests, run on every push
2. **CI pipeline**: integration tests (Failsafe), security scans, performance checks
3. **Production**: synthetic monitors, running continuously against the live API

The capstone's JUnit5 suite occupies layer one, the same layer a real pipeline runs first.

The Layers, Visualized



Contract Testing

Beyond the status code

KavinSchool

Contract Testing: Discussion

Your CI passes: the build compiles, unit tests are green.

Every endpoint returns `200 OK`.

A consumer's integration still breaks in production the same day.

> **DISCUSS**
>
> What did your pipeline never actually check?

Contract Testing: The Answer

A contract test checks the *shape* of a response: field names, types, and presence, against what consumers were promised.

- A renamed field, an undeclared extra field, or a changed type is invisible to a status-code check
- It is exactly what breaks a consumer in production

Contract Testing (Fintech)

REAL-WORLD SCENARIO

Fintech (Stripe — API Versioning):

- Stripe relies on consumer-driven contract testing to ensure changes to their payments API never break the payload shapes that millions of existing merchants rely on.
- A single missing field in a refund object would trigger a contract failure in the pipeline, even if the underlying code is fully operational.

Contract Testing (Ecommerce)

REAL-WORLD SCENARIO

Ecommerce (Wayfair — Search Services):

- Wayfair employs contract tests between their search APIs and frontend web clients.
- This guarantees that a renamed product attribute doesn't silently break the shopping experience, catching the error long before a 200 OK status hides the bug in production.

All Green, Still Broken

CI passes. Every endpoint returns `200 OK`. A consumer still breaks. What is the culprit?

A) Server crashed between CI and production

B) Response field was renamed, or its type changed

C) Database connection pool was exhausted

D) Consumer forgot to update their API key

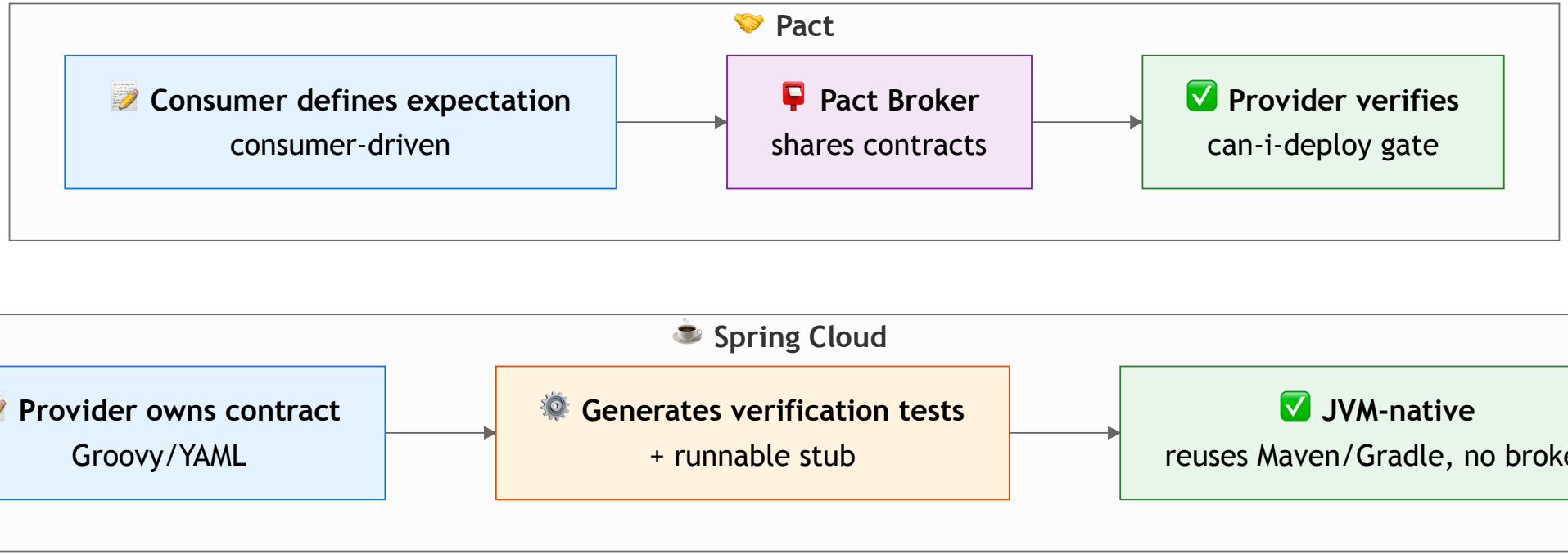
> 💬 **Chat Check**
>
> **Type your answer (A, B, C, or D) in the chat window now!**

All Green, Still Broken: The Answer

B, A status-code check says nothing about field names, types, or presence.

A contract test checks the *shape* of a response against what consumers were promised. A renamed field, an undeclared extra field, or a changed type is invisible to `200 OK`, and it's exactly what breaks a consumer in production.

Two Ways to Verify a Contract



Which Tool Fits This Stack

	Spring Cloud Contract	Pact
Model	Provider-owned, reuses Maven/Gradle	Consumer-driven, needs a broker
Best fit	All-JVM shops like this course's stack	Polyglot environments

Spring Cloud Contract is the more natural mention here; Pact is worth knowing by name.

Security Within the CI/CD Ecosystem

Where Day 3's controls plug into the pipeline

KavinSchool

Security: Where It Plugs In

- **Pre-commit**: SAST (Static Application Security Testing) and secrets scanning, before code is even pushed
- **CI (pull request)**: SAST and SCA (Software Composition Analysis / dependency scanning) gate the build
- **CD (before registry push)**: container image scans
- **Post-deploy**: DAST (Dynamic Application Security Testing) and runtime monitoring, usually nightly since DAST is far slower than SAST

> Day 3 already covered authN, authZ, and rate limiting - this is only about where those controls sit.

The Case for Deep API Knowledge

Beyond the endpoint

KavinSchool

The Cost of Shallow Practice

Postmortems on API outages consistently find the costliest failures rarely trace to one buggy line of code.

- They trace to ambiguous, poorly defined, or unenforced contracts
- A single overlooked assumption compounds once traffic scales
- Endpoint-only thinking shows up as inconsistency, vulnerability, and low consumer trust

The Payoff

Engineers who understand the whole lifecycle, design, versioning, security, and testing, reason about trade-offs directly instead of waiting on separate teams for each concern.

> That is why the course built one shared API across all four days instead of teaching each topic in isolation. The card applications capstone is where you rehearse the complete lifecycle.

Your Monday Morning

After four days of API development, what is one practice or mindset you will apply differently on Monday?

> **Chat Waterfall**
>
> **Type your thoughts in the chat, but DO NOT hit enter yet.**
>
> *Wait for my countdown: 3... 2... 1... BLAST*

Your Monday Morning (1/2)

If you mentioned any of these, the course did its job:

- **Write the contract first**, before a line of server code
- **Use standard status codes**, never `200` with an error body
- **Version with a plan**, soft deprecation, then hard, then sunset
- **AuthN is not authZ**, two different questions

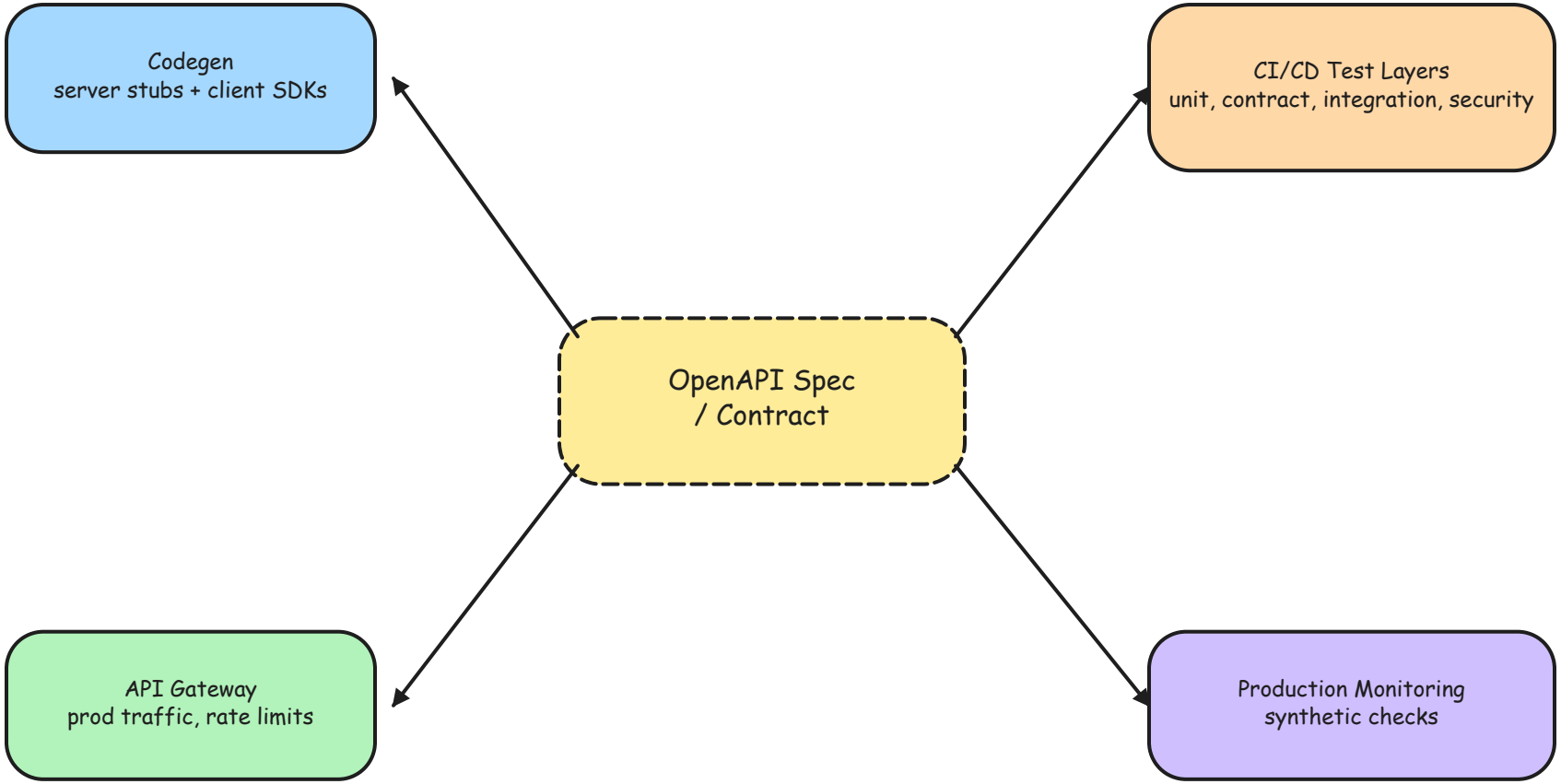
Your Monday Morning (2/2)

If you mentioned any of these, the course did its job:

- **Rate limit with a shared store**, not per-instance memory
- **Contract test the shape**, `200 OK` is not enough
- **You build it, you run it**, own what you ship

> 💬 **Pat on the back if you named any of these**

The Ecosystem, At a Glance



Card applications capstone — where it all comes together

Demo: Contract Drift Detector

Run `java -cp target/classes com.kavinschool.demos.DemoContractDriftDetector` from `day4/demos`.

- Compares sample responses against a small spec's expected field names and types
- Reports a renamed field, an undeclared extra field, and a changed type as drift
- A contract test asks "does the shape still match," not "did it return 200"

> Offline, no API key, no network.

Demo: Test Pyramid Runner

Run `java -cp target/classes com.kavinschool.demos.DemoTestPyramidRunner` from `day4/demos`.

- Runs unit, then integration, then contract layers, cheapest and fastest first
- A failing layer stops the run immediately, skipping the expensive layers below it
- A unit failure reports in ~3 seconds instead of ~40

> Offline, no API key, no network.

Card Applications API Capstone

Open `quickstart-project/breakouts/capstone/` in `api-dev-setup`.

Goal: Evolve, retire, secure, limit, document, and test one credit union API.

1. Compare the `v1`, `v2`, and `v3` contracts and identify breaking changes
2. Configure Spring Security and enforce the per-caller rate limit
3. Make all seven JUnit5/MockMvc acceptance tests pass

> The 60-minute capstone includes basic, intermediate, and stretch goals plus bounded LLM prompts.

Review of Capstone Solutions & Discussion

Same discipline, aimed at your own code

KavinSchool

Review of Capstone Solutions & Discussion

Discuss with your group, using evidence from the card applications capstone:

1. Which contract change truly required a new version, and which did not?
2. Where did request order change the observed `401`, `403`, or `429`?
3. What did an acceptance test catch that code inspection missed?

> Running behind? Compare `start/` against `solution/` one TODO at a time.

Key Takeaways

1. API-first turns the specification into an enforceable pipeline gate rather than passive documentation.
2. Reverse proxies, load balancers, and gateways have different responsibilities even when products overlap.
3. Layered testing runs the cheapest checks first, and contract tests verify response shape.
4. Correlated logs, metrics, and traces connect one request to production monitoring.
5. Costliest API failures trace to ambiguous or unenforced contracts.

Course Wrap-Up

Four days, one API, one running argument

KavinSchool

The Four-Day Arc, Completed

Day	Deliverable
1	Protocol and standards analysis of a real API
2	OpenAPI spec, peer reviewed, with interactive docs
3	Non-breaking versioning strategy and security rules
4	Versioned, secured, documented, and tested API capstone

Every row above is now checked off, on the same Spring Boot API, by you.

Back to Where We Started

On Day 1, each of you named one thing about APIs you wanted answered by today.

> **NOTE**
>
> Revisit the Day 1 introductions list now, one name at a time.

The tools will keep changing; the habit underneath all of them, evidence over assumption, is the one worth keeping.

Questions?

Thank you for four days of sharp judgment calls

KavinSchool